

ASSIGNMENT – 3

Breast Cancer Classification using Support Vector Machine

AIM

To implement a Support Vector Machine (SVM) classifier for breast cancer diagnosis using a real-world breast cancer prediction dataset (breast_cancer_prediction.csv), evaluate classification performance with multiple kernels (linear, polynomial, RBF, sigmoid), tune hyperparameters using GridSearchCV with 5-fold stratified cross-validation, and compare the models using accuracy, precision, recall, F1 score, ROC-AUC, confusion matrices, and ROC curves. The target variable is Biopsy_Result (Benign / Malignant), and the experiment also includes data inspection, missing-value imputation, standardization, and misclassification analysis. Libraries used: pandas, numpy, matplotlib, seaborn, scikit-learn.

DATASET DESCRIPTION

The Breast Cancer Prediction dataset consists of 10,000 patient records with clinical and diagnostic attributes (21 columns: demographics like Age, Gender and BMI, lifestyle factors like Smoking, Alcohol_Consumption and Physical_Activity, clinical measurements like Tumor_Size_cm, Blood_Pressure and Cholesterol, and an annual income field). The target variable is Biopsy_Result (Benign / Malignant), making this a binary classification problem. Patient_ID is a unique identifier column and carries no predictive information.

DATA PREPROCESSING

Missing values, irrelevant features, encoding, and standardization

Missing values: BMI, Tumor_Size_cm, Hormone_Therapy, Physical_Activity and Alcohol_Consumption each have 300 missing values (~3% each); all other columns have none. Missing numerical values are imputed with the median of each column computed only from the training set, then applied to both train and test, because the median is robust to outliers in skewed clinical measurements. Fitting the imputation on the training set only avoids data leakage from the test set.

Irrelevant features: Patient_ID is unique for every row (10000 unique values), so it is removed; all other columns are retained since none is constant or purely identifier-like.

Encoding and standardization: the categorical (object) columns are one-hot encoded with `pd.get_dummies(drop_first=True)` because they are nominal categories with no natural order. After encoding, the feature set has 22 columns. The data is split 80-20 (stratified on the target) into 8000 training and 2000 test samples, and `StandardScaler` is applied (fit on train, transform both) because SVM is a distance-based algorithm and unscaled features would dominate the margin computation.

CODE

```
import os
import warnings
import time

import matplotlib
matplotlib.use("Agg") # no pop-up windows; safe for capture_run
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

from sklearn.model_selection import (
    train_test_split, GridSearchCV, StratifiedKFold, cross_validate
)
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_curve, roc_auc_score
)

warnings.filterwarnings("ignore")

def main():
    # load the breast cancer prediction dataset
    script_dir = os.path.dirname(os.path.abspath(__file__))
    data_path = os.path.join(script_dir, "breast_cancer_prediction.csv")
    df = pd.read_csv(data_path)

    print("=" * 60)
    print("BREAST CANCER CLASSIFICATION USING SVM")
    print("=" * 60)

    print("\nDataset shape:", df.shape)
    print("\nFirst 5 rows:")
    print(df.head())

    # summary statistics of the dataset
    print("\nDataset info:")
    print(df.info())
    print("\nSummary statistics:")
    print(df.describe().to_string())

    # check missing values before preprocessing
    print("\nMissing values per column:")
    print(df.isnull().sum().sort_values(ascending=False).head(10))

    print("\nPercentage missing per column:")
    print((df.isnull().sum() / len(df) * 100).round(2).sort_values(ascending=False).head(10))

    print("\nUnique values per column:")
```

```
print(df.nunique())

print("\nIs Patient_ID unique for every row?", df["Patient_ID"].is_unique)

# categorical columns preview
cat_cols = df.drop(columns=["Biopsy_Result", "Patient_ID"]).select_dtypes(include=["object"]).columns
print("\nCategorical columns:", list(cat_cols))
for col in cat_cols:
    print(f" {col}: {df[col].nunique()} categories")

print("\nTarget class distribution:")
print(df["Biopsy_Result"].value_counts())

print("\nTarget class proportion:")
print(df["Biopsy_Result"].value_counts(normalize=True).round(3))

# EDA: class distribution bar chart
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="Biopsy_Result")
plt.title("Class Distribution: Benign vs Malignant")
plt.xlabel("Biopsy Result")
plt.ylabel("Count")
plt.show()

# EDA: correlation heatmap of numerical features
numeric_df = df.select_dtypes(include=["int64", "float64"])
numeric_df = numeric_df.drop(columns=["Patient_ID"])
plt.figure(figsize=(10, 7))
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap of Numerical Features")
plt.show()

# EDA: histograms of important numerical features
important_features = ["Age", "BMI", "Tumor_Size_cm", "Blood_Pressure", "Cholesterol"]
important_features = [f for f in important_features if f in df.columns]
for feature in important_features:
    plt.figure(figsize=(6, 4))
    sns.histplot(data=df, x=feature, kde=True)
    plt.title(f"Distribution of {feature}")
    plt.show()

# EDA: count of records for each categorical column (one grid figure)
fig, axes = plt.subplots(4, 3, figsize=(15, 12))
axes = axes.flatten()
for i, col in enumerate(cat_cols):
    sns.countplot(data=df, x=col, ax=axes[i])
    axes[i].set_title(f"Count of Records by {col}")
    axes[i].tick_params(axis="x", rotation=30)
for j in range(len(cat_cols), len(axes)):
    axes[j].axis("off")
plt.tight_layout()
plt.show()
```

```
# EDA: scatter plots colored by diagnosis
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
sns.scatterplot(data=df, x="Age", y="Tumor_Size_cm", hue="Biopsy_Result", alpha=0.5, ax=axes[0])
axes[0].set_title("Tumor Size vs Age by Biopsy Result")
sns.scatterplot(data=df, x="BMI", y="Cholesterol", hue="Biopsy_Result", alpha=0.5, ax=axes[1])
axes[1].set_title("Cholesterol vs BMI by Biopsy Result")
plt.tight_layout()
plt.show()

# EDA: box plots of important features
fig, axes = plt.subplots(1, len(important_features), figsize=(15, 4))
for ax, feature in zip(axes, important_features):
    sns.boxplot(data=df, y=feature, ax=ax)
    ax.set_title(f"Box Plot of {feature}")
plt.tight_layout()
plt.show()

# EDA: box plots of each feature split by diagnosis
fig, axes = plt.subplots(1, len(important_features), figsize=(15, 4))
for ax, feature in zip(axes, important_features):
    sns.boxplot(data=df, x="Biopsy_Result", y=feature, ax=ax)
    ax.set_title(f"{feature} by Biopsy Result")
plt.tight_layout()
plt.show()

# EDA: pair plot of important features (sampled to keep it fast)
pairplot_sample = df.sample(n=min(500, len(df)), random_state=42)
sns.pairplot(pairplot_sample, vars=important_features, hue="Biopsy_Result", diag_kind="hist")
plt.show()

# separate features and target
X = df.drop(columns=["Biopsy_Result", "Patient_ID"])
y = df["Biopsy_Result"].map({"Benign": 0, "Malignant": 1})

# one-hot encode categorical features
categorical_columns = X.select_dtypes(include=["object"]).columns
X = pd.get_dummies(X, columns=categorical_columns, drop_first=True).astype(float)

# train-test split with stratification
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# impute missing values using training set medians only
train_medians = X_train.median()
X_train = X_train.fillna(train_medians)
X_test = X_test.fillna(train_medians)

# standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)

print("\nTraining set shape:", X_train.shape)
print("Test set shape:", X_test.shape)
print("Missing values after imputation:", X_train.isnull().sum().sum())

# baseline SVM with four kernels
print("\n" + "=" * 60)
print("SVM BASELINE RESULTS")
print("=" * 60)

kernels = ["linear", "poly", "rbf", "sigmoid"]
basic_results = []
for kernel in kernels:
    start = time.time()
    model = SVC(
        kernel=kernel, C=1, gamma="scale", degree=3,
        probability=True, random_state=42
    )
    model.fit(X_train_scaled, y_train)
    train_time = time.time() - start

    start = time.time()
    pred = model.predict(X_test_scaled)
    pred_time = time.time() - start

    basic_results.append({
        "Kernel": kernel.capitalize(),
        "Training Accuracy": accuracy_score(y_train, model.predict(X_train_scaled)),
        "Testing Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred, zero_division=0),
        "Recall": recall_score(y_test, pred, zero_division=0),
        "F1 Score": f1_score(y_test, pred, zero_division=0),
        "Training Time (s)": train_time,
        "Prediction Time (s)": pred_time,
    })

basic_results_df = pd.DataFrame(basic_results)
print(basic_results_df.to_string(index=False))

# hyperparameter tuning with reduced search space
print("\n" + "=" * 60)
print("HYPERPARAMETER TUNING")
print("=" * 60)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
C_values = [1, 10]
gamma_values = ["scale", 0.01]
degree_values = [2, 3]

searches = {
    "Linear": GridSearchCV(
```

```
SVC(kernel="linear", random_state=42),
{"C": C_values}, cv=cv, scoring="accuracy", n_jobs=-1
),
"Polynomial": GridSearchCV(
    SVC(kernel="poly", random_state=42),
    {"C": C_values, "gamma": gamma_values, "degree": degree_values},
    cv=cv, scoring="accuracy", n_jobs=-1
),
"RBF": GridSearchCV(
    SVC(kernel="rbf", random_state=42),
    {"C": C_values, "gamma": gamma_values},
    cv=cv, scoring="accuracy", n_jobs=-1
),
"Sigmoid": GridSearchCV(
    SVC(kernel="sigmoid", random_state=42),
    {"C": C_values, "gamma": gamma_values},
    cv=cv, scoring="accuracy", n_jobs=-1
),
}

for name, search in searches.items():
    search.fit(X_train_scaled, y_train)

# refit best models with probability=True for ROC curves
best_models = {}
for name, search in searches.items():
    best_params = search.best_params_
    kernel_name = search.estimator.kernel
    refit_model = SVC(
        kernel=kernel_name, probability=True, random_state=42, **best_params
    )
    refit_model.fit(X_train_scaled, y_train)
    best_models[name] = refit_model

tuning_summary = []
for name, search in searches.items():
    tuning_summary.append({
        "Kernel": name,
        "Best C": search.best_params_.get("C", "NA"),
        "Best Gamma": search.best_params_.get("gamma", "NA"),
        "Best Degree": search.best_params_.get("degree", "NA"),
        "Best CV Accuracy": search.best_score_,
    })

tuning_summary_df = pd.DataFrame(tuning_summary)
print("\nTable 1: Hyperparameter Tuning Summary")
print(tuning_summary_df.to_string(index=False))

# final predictions and probabilities on test set
print("\n" + "=" * 60)
print("FINAL MODEL RESULTS")
print("=" * 60)
```

```
predictions = {}
probabilities = {}
train_times = {}
pred_times = {}

for name, model in best_models.items():
    start = time.time()
    model.fit(X_train_scaled, y_train)
    train_times[name] = time.time() - start

    start = time.time()
    predictions[name] = model.predict(X_test_scaled)
    pred_times[name] = time.time() - start

    probabilities[name] = model.predict_proba(X_test_scaled)[: , 1]

# confusion matrices
fig, axes = plt.subplots(1, 4, figsize=(18, 4))
for ax, (name, pred) in zip(axes, predictions.items()):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(
        cm, annot=True, fmt="d", cmap="Blues",
        xticklabels=["Benign", "Malignant"],
        yticklabels=["Benign", "Malignant"], ax=ax
    )
    ax.set_title(f"{name} Kernel")
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
plt.suptitle("Confusion Matrices of Tuned SVM Kernels", y=1.02)
plt.tight_layout()
plt.show()

# ROC curves
plt.figure(figsize=(7, 6))
for name, prob in probabilities.items():
    fpr, tpr, _ = roc_curve(y_test, prob)
    auc_score = roc_auc_score(y_test, prob)
    plt.plot(fpr, tpr, label=f"{name} (AUC = {auc_score:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Random Guess")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()
plt.show()

# accuracy vs F1 score comparison
all_kernels = list(predictions.keys())
acc_scores = [accuracy_score(y_test, predictions[k]) for k in all_kernels]
f1_scores = [f1_score(y_test, predictions[k], zero_division=0) for k in all_kernels]

x = np.arange(len(all_kernels))
```

```
width = 0.35
plt.figure(figsize=(7, 5))
plt.bar(x - width / 2, acc_scores, width, label="Accuracy")
plt.bar(x + width / 2, f1_scores, width, label="F1 Score")
plt.xticks(x, all_kernels)
plt.xlabel("Kernel")
plt.ylabel("Score")
plt.title("Kernel Comparison: Accuracy vs F1 Score")
plt.legend()
plt.ylim(0, 1)
plt.show()

# classification reports
print("\nClassification Reports:")
for name, pred in predictions.items():
    print(f"\n{name} Kernel")
    print(classification_report(y_test, pred, target_names=["Benign", "Malignant"]))

# cross-validation performance on training set
print("\n" + "=" * 60)
print("CROSS-VALIDATION PERFORMANCE (5-FOLD)")
print("=" * 60)

scoring = {
    "accuracy": "accuracy",
    "precision": "precision",
    "recall": "recall",
    "f1": "f1",
}

cv_results = []
for name, model in best_models.items():
    scores = cross_validate(
        model, X_train_scaled, y_train, cv=cv, scoring=scoring, n_jobs=-1
    )
    cv_results.append({
        "Kernel": name,
        "Accuracy": scores["test_accuracy"].mean(),
        "Precision": scores["test_precision"].mean(),
        "Recall": scores["test_recall"].mean(),
        "F1 Score": scores["test_f1"].mean(),
    })

cv_df = pd.DataFrame(cv_results)
print(cv_df.to_string(index=False))

# test set performance
print("\n" + "=" * 60)
print("TEST SET PERFORMANCE")
print("=" * 60)

test_results = []
for name, model in best_models.items():
```

```
pred = predictions[name]
prob = probabilities[name]
test_results.append({
    "Kernel": name,
    "Training Accuracy": accuracy_score(y_train, model.predict(X_train_scaled)),
    "Testing Accuracy": accuracy_score(y_test, pred),
    "Precision": precision_score(y_test, pred, zero_division=0),
    "Recall": recall_score(y_test, pred, zero_division=0),
    "F1 Score": f1_score(y_test, pred, zero_division=0),
    "ROC-AUC": roc_auc_score(y_test, prob),
    "Training Time (s)": train_times[name],
    "Prediction Time (s)": pred_times[name],
})

test_df = pd.DataFrame(test_results)
print(test_df.to_string(index=False))

# best kernel
best_kernel_row = test_df.loc[test_df["Testing Accuracy"].idxmax()]
best_kernel_name = best_kernel_row["Kernel"]
print("\nBest performing kernel based on testing accuracy:")
print(best_kernel_row.to_string())

# some misclassified samples
best_pred = predictions[best_kernel_name]
misclassified_mask = best_pred != y_test.values
misclassified_indices = y_test.index[misclassified_mask]
misclassified_df = df.loc[misclassified_indices].copy()
misclassified_df["Predicted"] = pd.Series(
    best_pred, index=y_test.index
)[misclassified_mask].map({0: "Benign", 1: "Malignant"})

print("\nNumber of misclassified samples:", len(misclassified_df))
print(misclassified_df.head().to_string())

print("\nExperiment completed.")

if __name__ == "__main__":
    main()
```

OUTPUT

```
=====
BREAST CANCER CLASSIFICATION USING SVM
=====

Dataset shape: (10000, 21)

First 5 rows:
   Patient_ID  Age  ... Breastfeeding_History  Annual_Income_USD
0    100001    71  ...                      No                101316
1    100002    34  ...                      Yes                 30023
```

```
2      100003  80 ...      Yes      34819
3      100004  40 ...      Yes      68025
4      100005  43 ...      Yes      90919
```

```
[5 rows x 21 columns]
```

Dataset info:

```
<class 'pandas.DataFrame'>
```

RangeIndex: 10000 entries, 0 to 9999

Data columns (total 21 columns):

#	Column	Non-Null Count	Dtype
0	Patient_ID	10000 non-null	int64
1	Age	10000 non-null	int64
2	Gender	10000 non-null	str
3	BMI	9700 non-null	float64
4	Family_History	10000 non-null	str
5	Smoking	10000 non-null	str
6	Alcohol_Consumption	9700 non-null	str
7	Physical_Activity	9700 non-null	str
8	Hormone_Therapy	9700 non-null	str
9	Menopause_Status	10000 non-null	str
10	Genetic_Mutation	10000 non-null	str
11	Tumor_Size_cm	9700 non-null	float64
12	Lymph_Node_Involvement	10000 non-null	str
13	Mammogram_Result	10000 non-null	str
14	Biopsy_Result	10000 non-null	str
15	Blood_Pressure	10000 non-null	int64
16	Cholesterol	10000 non-null	int64
17	Diabetes	10000 non-null	str
18	Exercise_Days_Per_Week	10000 non-null	int64
19	Breastfeeding_History	10000 non-null	str
20	Annual_Income_USD	10000 non-null	int64

dtypes: float64(2), int64(6), str(13)

memory usage: 1.6 MB

None

Summary statistics:

	Patient_ID	Age	BMI	Tumor_Size_cm	Blood_Pressure	Cholesterol	Exercise_Days_Per_Week	Annual_Income_USD
count	10000.00000	10000.00000	9700.00000	9700.00000	10000.00000	10000.00000	10000.00000	10000.00000
mean	105000.50000	52.033800	27.051227	2.630334	134.811100	214.071500	3.535200	82337.305000
std	2886.89568	18.789517	4.494501	1.589789	26.174742	49.242034	2.282385	39102.629984
min	100001.00000	20.000000	11.100000	0.000000	90.000000	130.000000	0.000000	15002.000000
25%	102500.75000	36.000000	24.000000	1.370000	112.000000	172.000000	2.000000	48145.750000
50%	105000.50000	52.000000	27.000000	2.510000	135.000000	213.000000	4.000000	81813.500000
75%	107500.25000	68.000000	30.100000	3.720000	157.000000	257.000000	6.000000	116217.500000
max	110000.00000	84.000000	44.100000	9.880000	180.000000	300.000000	7.000000	149996.000000

Missing values per column:

BMI	300
Tumor_Size_cm	300
Hormone_Therapy	300
Physical_Activity	300
Alcohol_Consumption	300
Family_History	0
Gender	0
Age	0
Patient_ID	0
Smoking	0

dtype: int64

Percentage missing per column:

BMI	3.0
Tumor_Size_cm	3.0
Hormone_Therapy	3.0
Physical_Activity	3.0
Alcohol_Consumption	3.0
Family_History	0.0
Gender	0.0
Age	0.0
Patient_ID	0.0
Smoking	0.0

dtype: float64

Unique values per column:

Patient_ID	10000
------------	-------

```

Age                65
Gender              2
BMI                280
Family_History      2
Smoking             2
Alcohol_Consumption 2
Physical_Activity    3
Hormone_Therapy      2
Menopause_Status     2
Genetic_Mutation     2
Tumor_Size_cm       728
Lymph_Node_Involvement 2
Mammogram_Result     3
Biopsy_Result        2
Blood_Pressure       91
Cholesterol          171
Diabetes             2
Exercise_Days_Per_Week 8
Breastfeeding_History 3
Annual_Income_USD    9645
dtype: int64

```

Is Patient_ID unique for every row? True

Categorical columns: ['Gender', 'Family_History', 'Smoking', 'Alcohol_Consumption', 'Physical_Activity', 'Hormone_Therapy', 'Menopause_Status', 'Genetic_Mutation', 'Lymph_Node_Involvement', 'Mammogram_Result', 'Diabetes', 'Breastfeeding_History']

```

Gender: 2 categories
Family_History: 2 categories
Smoking: 2 categories
Alcohol_Consumption: 2 categories
Physical_Activity: 3 categories
Hormone_Therapy: 2 categories
Menopause_Status: 2 categories
Genetic_Mutation: 2 categories
Lymph_Node_Involvement: 2 categories
Mammogram_Result: 3 categories
Diabetes: 2 categories
Breastfeeding_History: 3 categories

```

Target class distribution:

```

Biopsy_Result
Benign      8067
Malignant   1933
Name: count, dtype: int64

```

Target class proportion:

```

Biopsy_Result
Benign      0.807
Malignant   0.193
Name: proportion, dtype: float64

```

Training set shape: (8000, 22)

Test set shape: (2000, 22)

Missing values after imputation: 0

SVM BASELINE RESULTS

Kernel	Training Accuracy	Testing Accuracy	Precision	Recall	F1 Score	Training Time (s)	Prediction Time (s)
Linear	0.939250	0.9350	0.839050	0.821705	0.830287	5.923644	0.094183
Poly	0.971625	0.9250	0.839542	0.757106	0.796196	5.891793	0.170393
Rbf	0.968250	0.9380	0.854447	0.819121	0.836412	3.615521	0.297402
Sigmoid	0.904000	0.9155	0.796196	0.757106	0.776159	2.270563	0.072771

HYPERPARAMETER TUNING

Table 1: Hyperparameter Tuning Summary

Kernel	Best C	Best Gamma	Best Degree	Best CV Accuracy
Linear	10	NA	NA	0.937250
Polynomial	1	scale	3	0.928625
RBF	1	scale	NA	0.937500
Sigmoid	10	0.01	NA	0.935625

FINAL MODEL RESULTS

=====

Classification Reports:

Linear Kernel

	precision	recall	f1-score	support
Benign	0.96	0.96	0.96	1613
Malignant	0.84	0.82	0.83	387
accuracy			0.94	2000
macro avg	0.90	0.89	0.90	2000
weighted avg	0.94	0.94	0.94	2000

Polynomial Kernel

	precision	recall	f1-score	support
Benign	0.94	0.97	0.95	1613
Malignant	0.84	0.76	0.80	387
accuracy			0.93	2000
macro avg	0.89	0.86	0.88	2000
weighted avg	0.92	0.93	0.92	2000

RBF Kernel

	precision	recall	f1-score	support
Benign	0.96	0.97	0.96	1613
Malignant	0.85	0.82	0.84	387
accuracy			0.94	2000
macro avg	0.91	0.89	0.90	2000
weighted avg	0.94	0.94	0.94	2000

Sigmoid Kernel

	precision	recall	f1-score	support
Benign	0.95	0.96	0.96	1613
Malignant	0.84	0.80	0.82	387
accuracy			0.93	2000
macro avg	0.90	0.88	0.89	2000
weighted avg	0.93	0.93	0.93	2000

=====

CROSS-VALIDATION PERFORMANCE (5-FOLD)

=====

Kernel	Accuracy	Precision	Recall	F1 Score
Linear	0.937250	0.856154	0.811769	0.833292
Polynomial	0.928625	0.868230	0.743848	0.801082
RBF	0.937500	0.874801	0.789784	0.830077
Sigmoid	0.935625	0.857422	0.800125	0.827638

=====

TEST SET PERFORMANCE

=====

Kernel	Training Accuracy	Testing Accuracy	Precision	Recall	F1 Score	ROC-AUC	Training Time (s)	Prediction Time (s)
Linear	0.939375	0.936	0.841689	0.824289	0.832898	0.975261	10.384583	0.031974
Polynomial	0.971625	0.925	0.839542	0.757106	0.796196	0.970096	2.247390	0.052105
RBF	0.968250	0.938	0.854447	0.819121	0.836412	0.975648	3.122798	0.289416
Sigmoid	0.933125	0.932	0.838275	0.803618	0.820580	0.974919	2.253876	0.074924

Best performing kernel based on testing accuracy:

Kernel	RBF
Training Accuracy	0.96825
Testing Accuracy	0.938
Precision	0.854447
Recall	0.819121
F1 Score	0.836412
ROC-AUC	0.975648
Training Time (s)	3.122798

Prediction Time (s) 0.289416

Number of misclassified samples: 124

Patient_ID	Age	Gender	BMI	Family_History	Smoking	Alcohol_Consumption	Physical_Activity	Hormone_Therapy	Menopause_Status	Genetic_Mutation	Tumor_Size_cm	Lymph_Node_Involvement	Mammogram_Result	Biopsy_Result	Blood_Pressure	Cholesterol	Diabetes	Exercise_Days_Per_Week	Breastfeeding_History	Annual_Income_USD	Predicted
1352	101353	25	Female	25.2	No	Yes	Yes	High	No	Pre	Positive	Not Applicable									
1.19			Yes	Abnormal	Malignant	128	164	Yes		6											
137189			Benign																		
1892	101893	57	Female	23.1	No	Yes	No	Low	No	Pre	Negative	Yes									
4.37			Yes	Normal	Malignant	126	265	No		7											
56273			Benign																		
9321	109322	46	Male	28.4	Yes	Yes	Yes	Moderate	No	Post	Positive	No									
2.62			No	Normal	Benign	103	207	No		6											
80431			Malignant																		
9658	109659	55	Female	35.0	No	No	Yes	Low	No	Post	Positive	Yes									
3.39			No	Normal	Malignant	166	212	No		1											
93443			Benign																		
1392	101393	54	Male	34.2	No	No	No	Moderate	Yes	Post	Negative	Yes									
3.03			Yes	Normal	Malignant	144	170	No		3											
113354			Benign																		

Experiment completed.

PLOTS

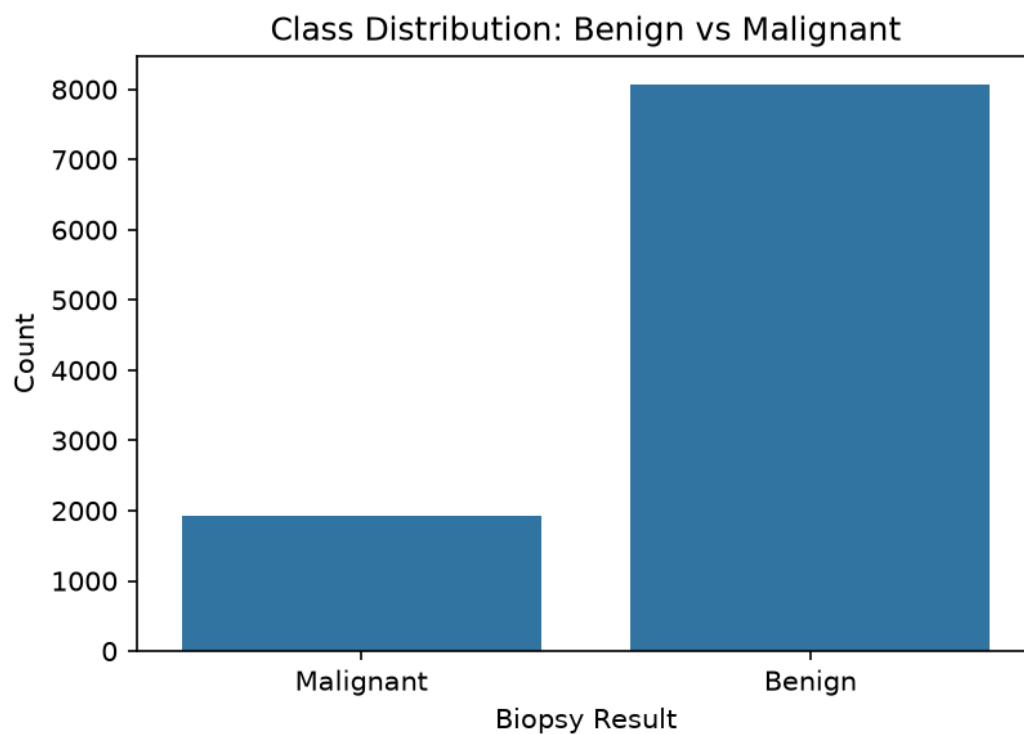


Fig 1 - Class Distribution: Benign vs Malignant

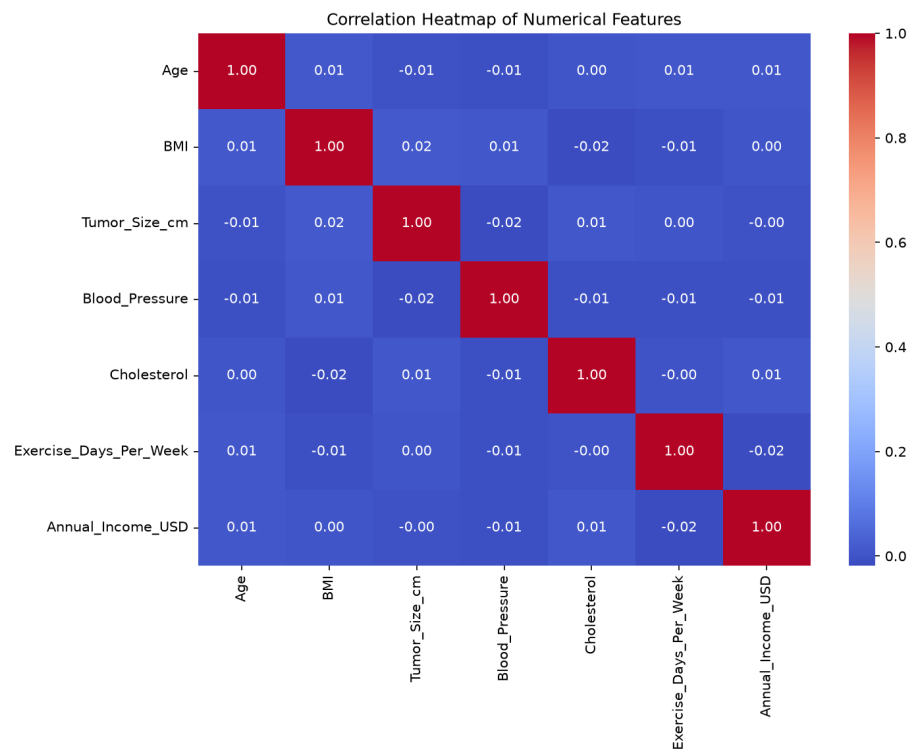


Fig 2 - Correlation Heatmap of Numerical Features

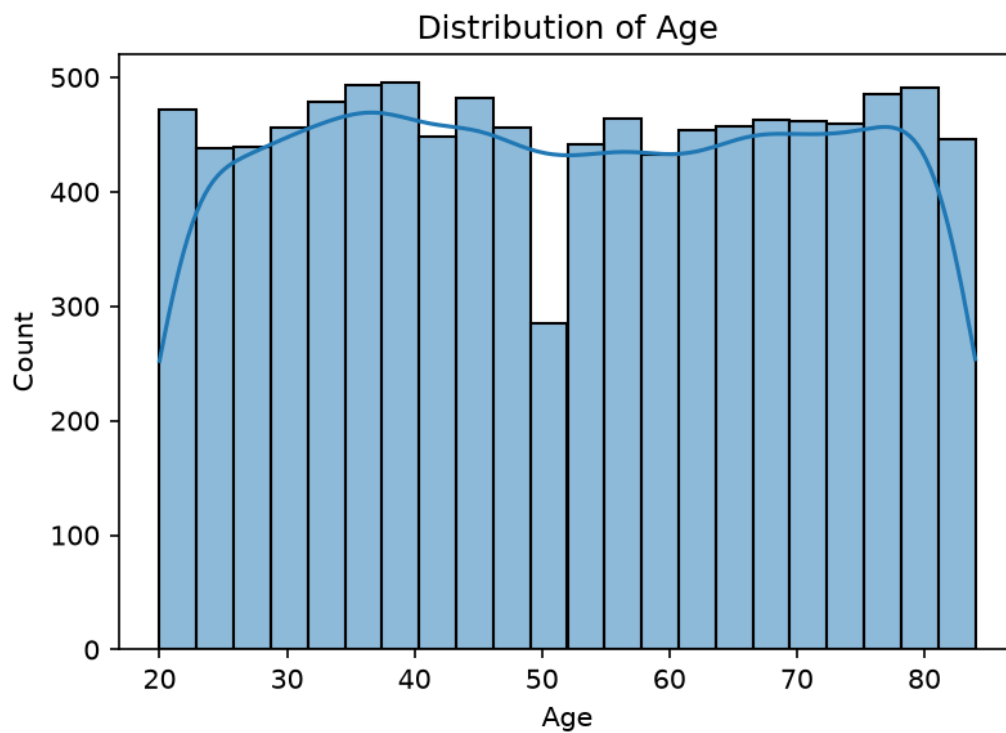


Fig 3 - Distribution of Age

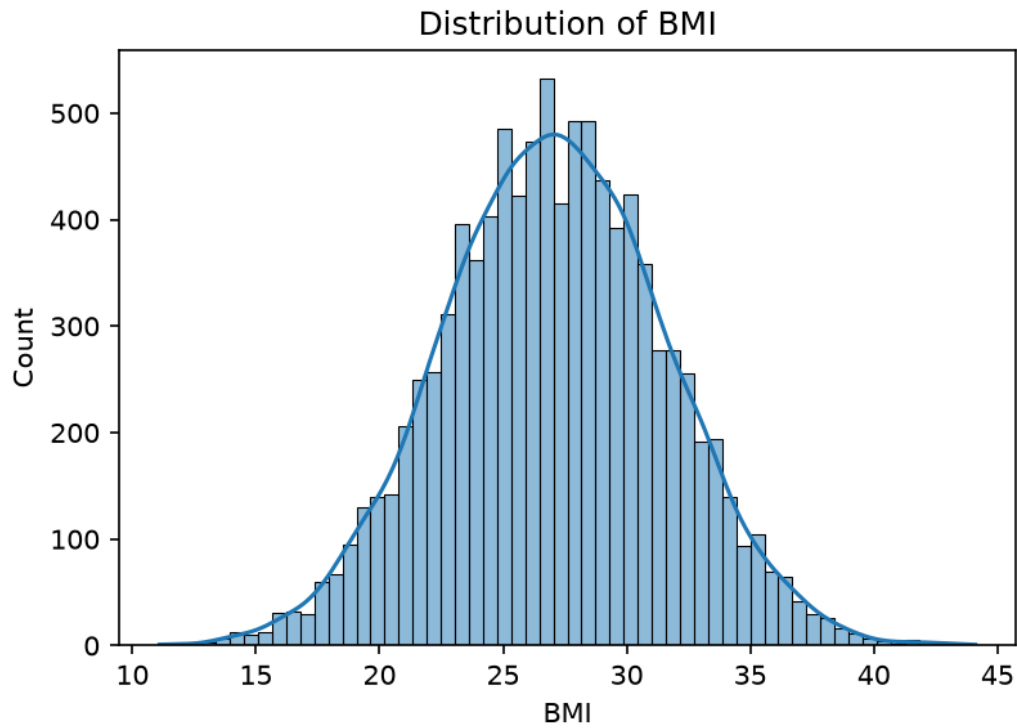


Fig 4 - Distribution of BMI

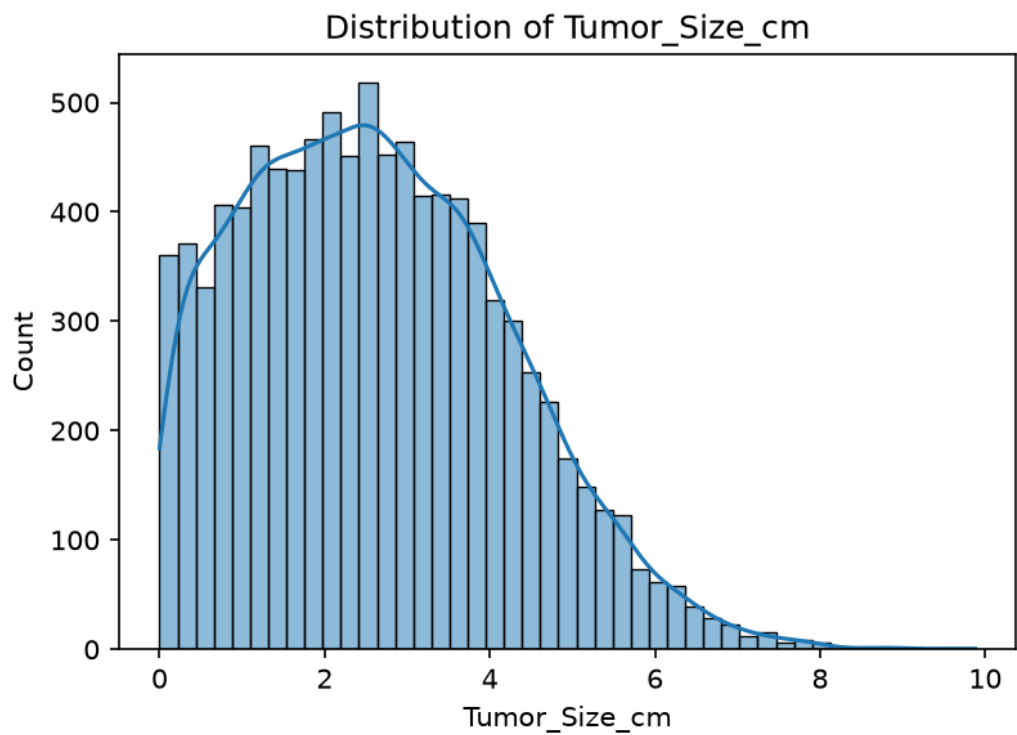


Fig 5 - Distribution of Tumor Size (cm)

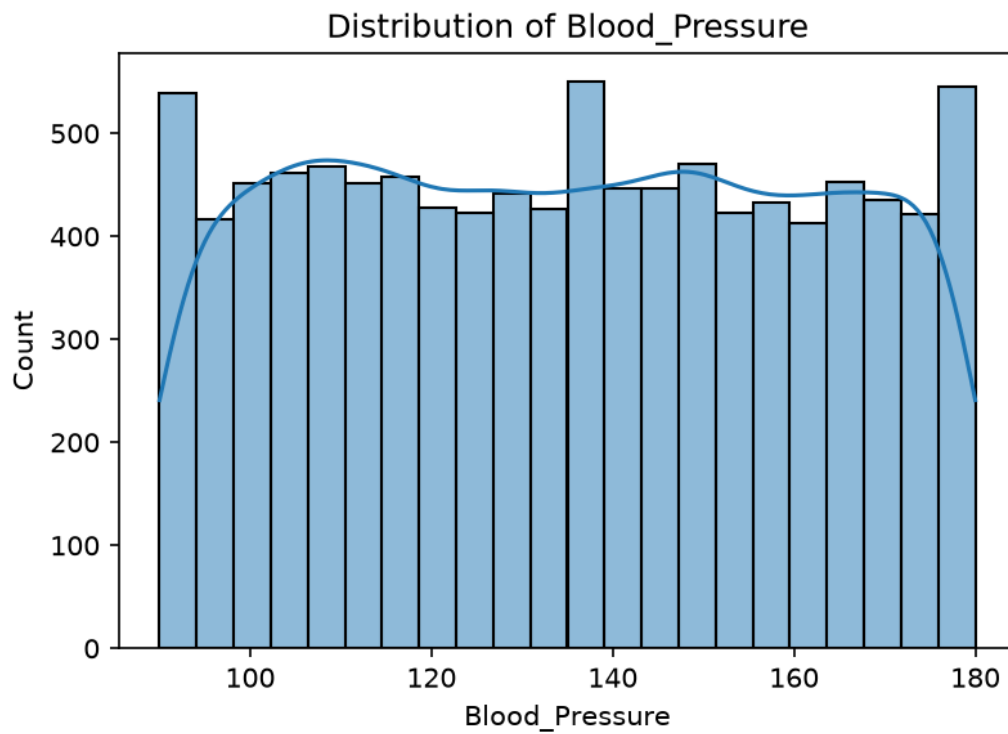


Fig 6 - Distribution of Blood Pressure

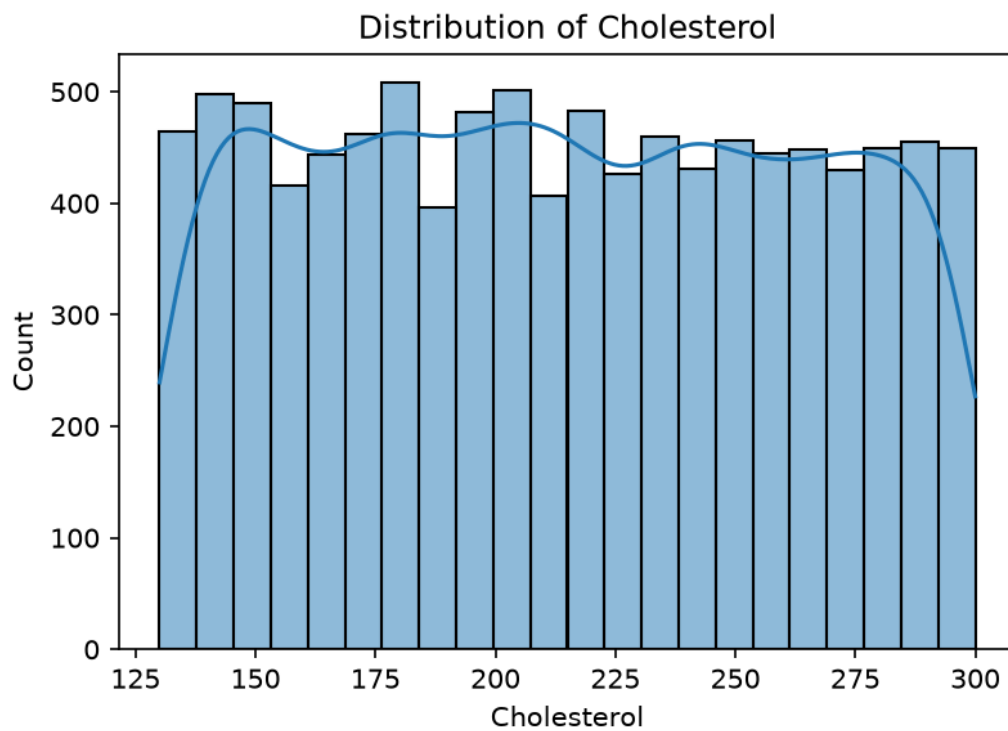


Fig 7 - Distribution of Cholesterol

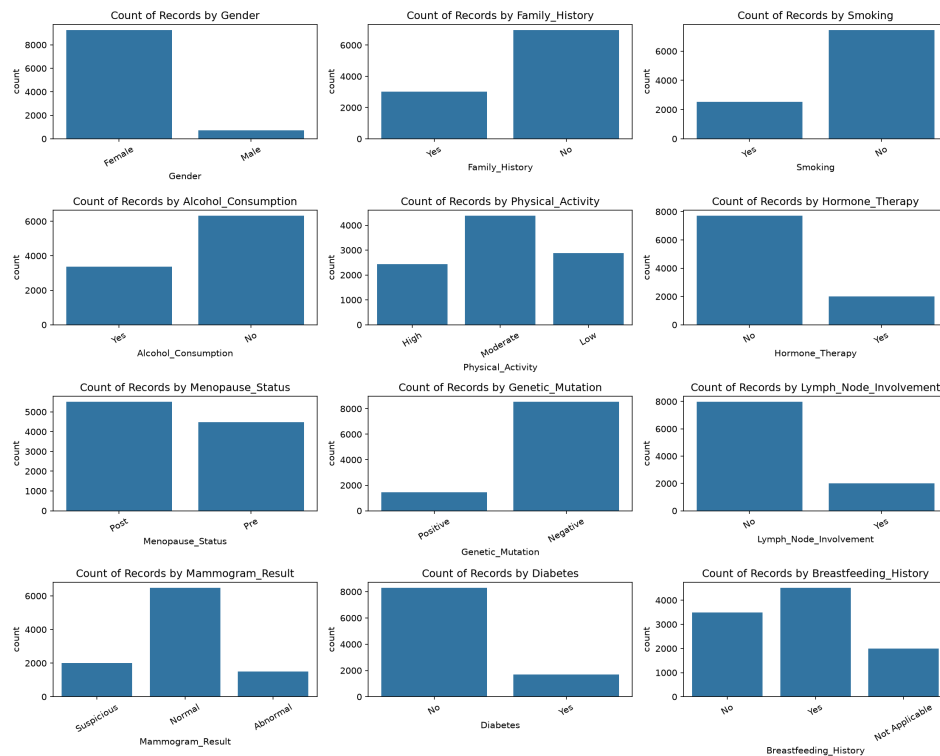


Fig 8 - Count of Records by Categorical Features

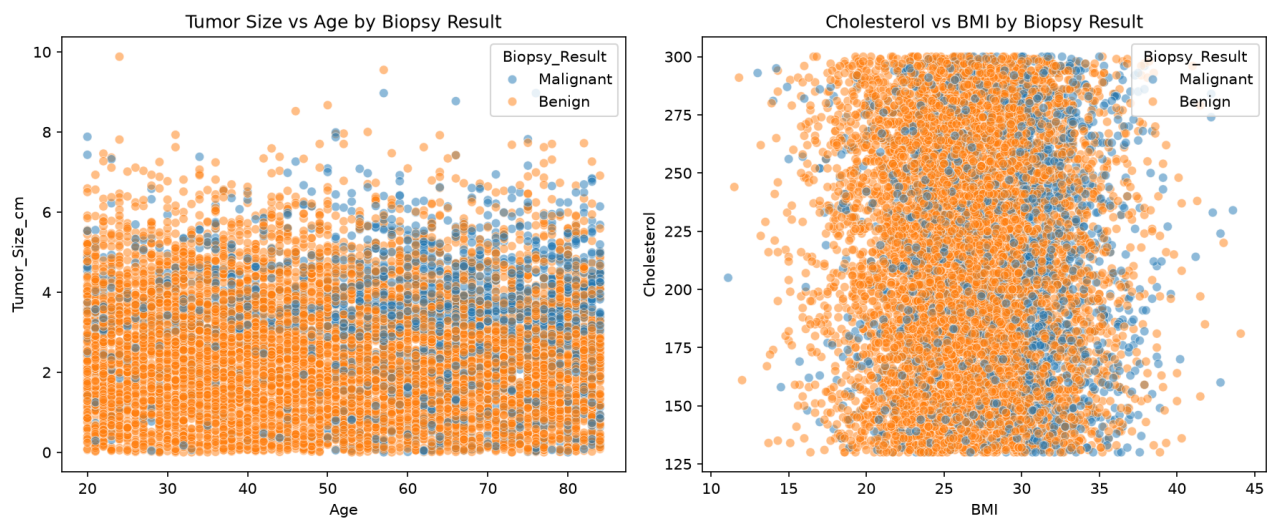


Fig 9 - Scatter Plots: Tumor Size vs Age and Cholesterol vs BMI

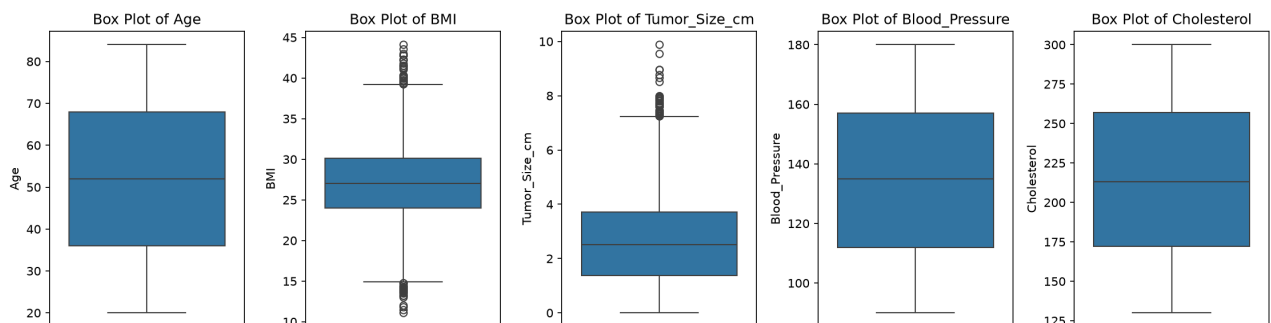


Fig 10 - Box Plots of Important Features

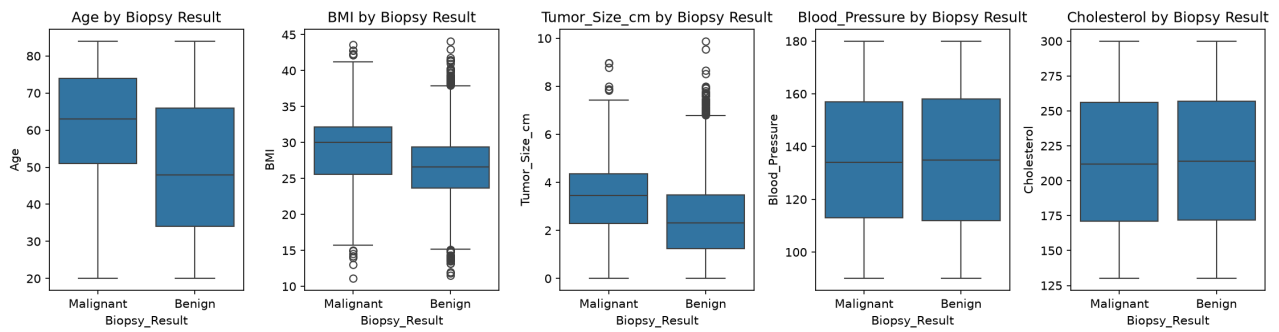


Fig 11 - Box Plots of Features Split by Diagnosis

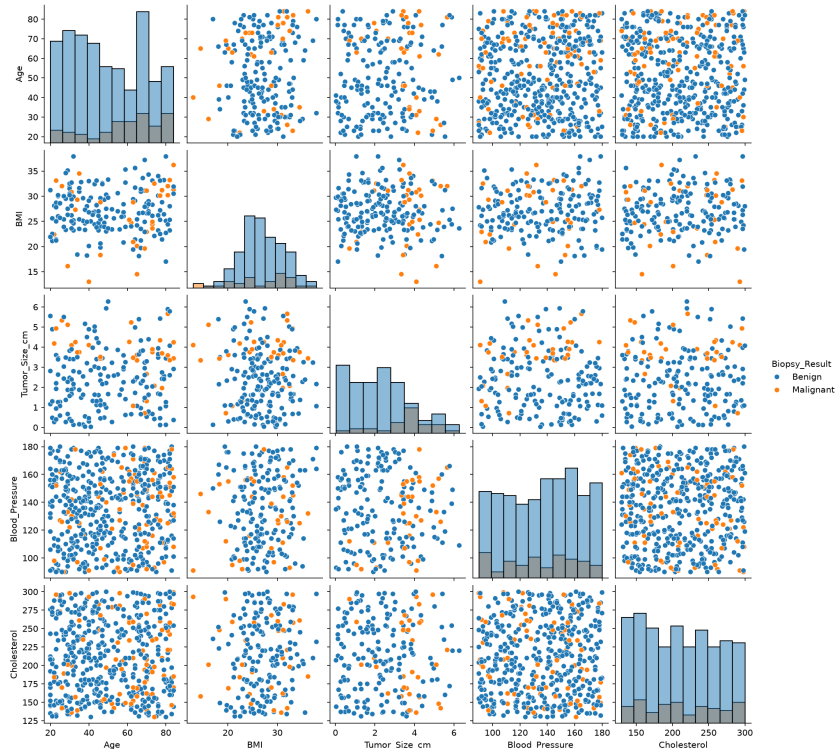


Fig 12 - Pair Plot of Important Features

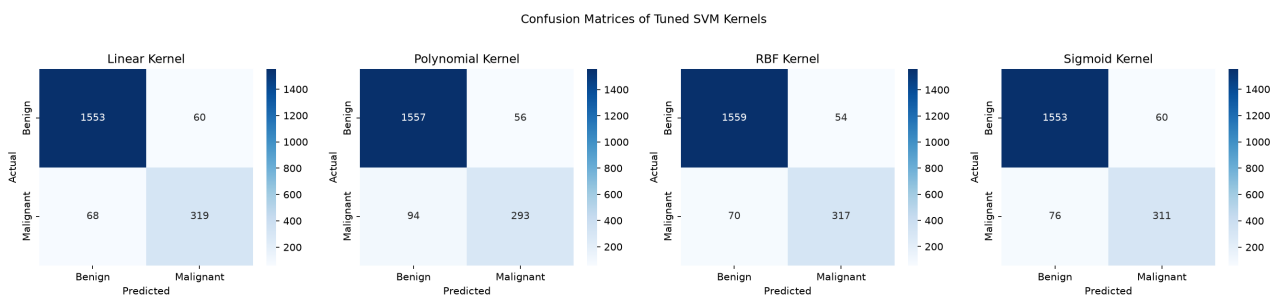


Fig 13 - Confusion Matrices of Tuned SVM Kernels

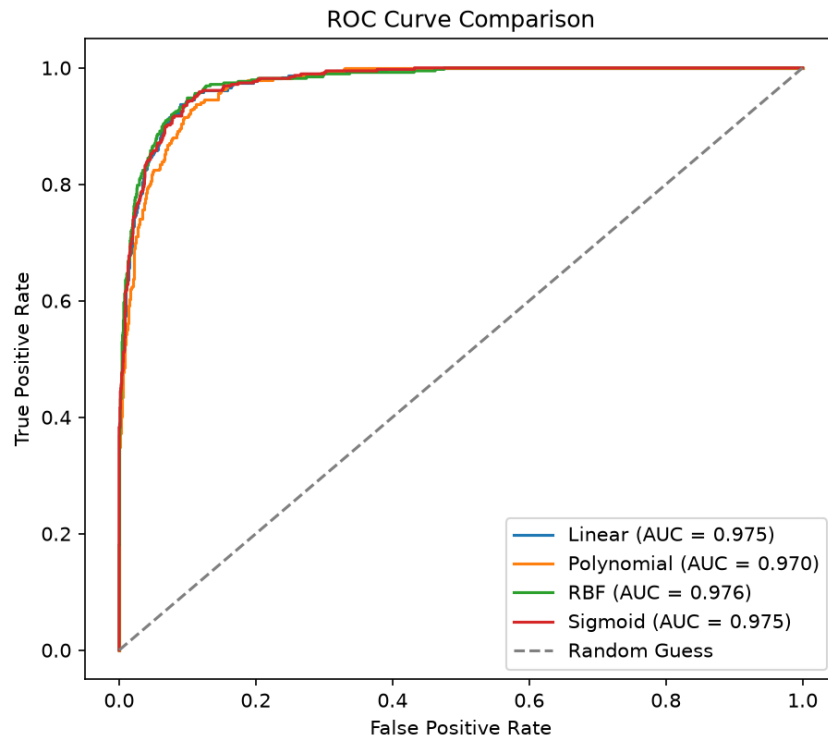


Fig 14 - ROC Curve Comparison

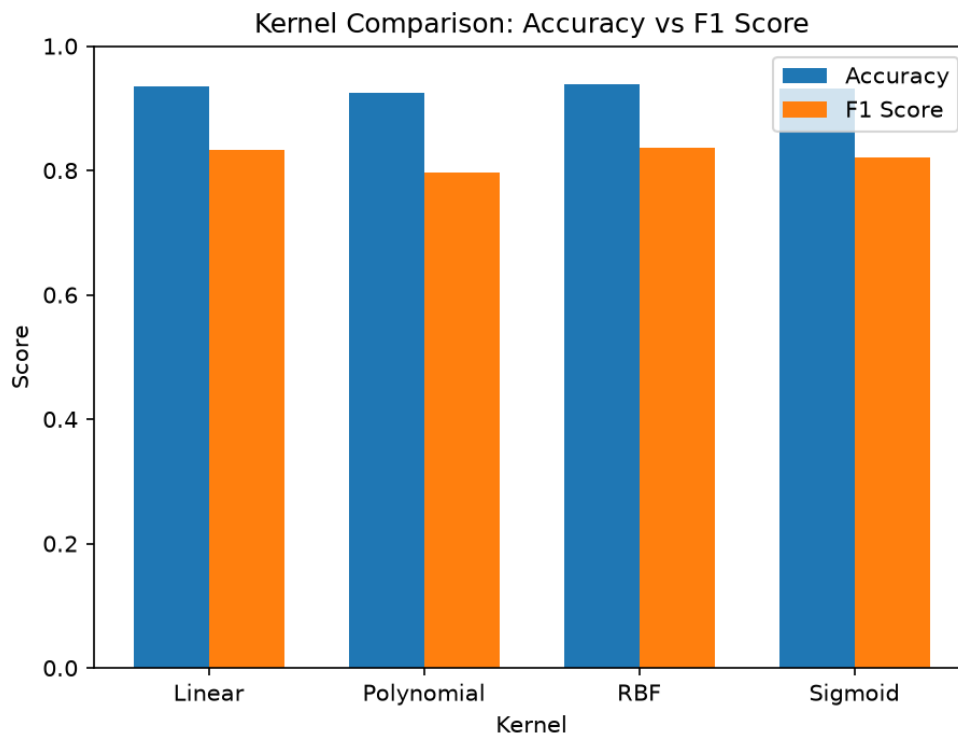


Fig 15 - Kernel Comparison: Accuracy vs F1 Score

TASKS PERFORMED

Task	Description	Result / Output
Loaded and inspected the dataset	Read breast_cancer_prediction.csv with pandas and checked shape, info, and describe	10000 rows x 21 columns; 8 int, 8 float, 5 object columns

Task	Description	Result / Output
Checked missing values	Used <code>isnull().sum()</code> and percentage missing per column	BMI, Tumor_Size_cm, Hormone_Therapy, Physical_Activity, Alcohol_Consumption each had 300 missing values (~3%); Family_History, Gender, Age, Patient_ID, Smoking had 0
Checked unique values	Used <code>nunique()</code> to inspect categorical / numerical cardinality	Patient_ID has 10000 unique values (removed); Biopsy_Result has 2 classes; Mammogram_Result has 3 categories
Explored target distribution	Count and proportion of Benign vs Malignant cases	Benign: 8067 (80.7%), Malignant: 1933 (19.3%) - imbalanced
Visualized EDA plots	Class distribution, correlation heatmap, histograms, categorical count bars, scatter plots, box plots, and pair plot	Figs 1-12
Prepared features and target	Dropped Patient_ID and Biopsy_Result; one-hot encoded categorical columns; mapped target to 0 (Benign) and 1 (Malignant)	Final feature set has 22 columns
Split the dataset	Stratified train-test split with <code>test_size=0.20</code> and <code>random_state=42</code>	Training: 8000 samples, Test: 2000 samples
Imputed missing values	Filled missing values using training-set medians for both train and test	0 missing values after imputation
Standardized features	StandardScaler fit on training set and applied to both train and test	Features scaled to mean 0 and std 1
Trained baseline SVM models	SVC with <code>C=1</code> , <code>gamma=scale</code> , <code>degree=3</code> , <code>probability=True</code> for all four kernels	Linear test acc 0.9350, Poly 0.9250, RBF 0.9380, Sigmoid 0.9155
Tuned hyperparameters	GridSearchCV with 5-fold StratifiedKFold over C, gamma, and degree for each kernel	Linear: C=10; Polynomial: C=1, gamma=scale, degree=3; RBF: C=1, gamma=scale; Sigmoid: C=10, gamma=0.01
Evaluated tuned models	Predictions, classification reports, confusion matrices, ROC-AUC on the test set	Figs 13-15; RBF achieved best test accuracy 0.938
Ran 5-fold cross-validation	<code>cross_validate</code> on the training set with accuracy, precision, recall, f1 scoring	RBF: 0.937500 accuracy, Linear: 0.937250, Sigmoid: 0.935625, Polynomial: 0.928625

Task	Description	Result / Output
Analyzed misclassifications	Collected test samples where best kernel (RBF) prediction differs from actual label	124 misclassified samples out of 2000 test rows

OBSERVATIONS

- Missing values are imputed with training-set medians only, avoiding data leakage.
- Patient_ID is removed as an irrelevant identifier feature.
- Categorical features are one-hot encoded before model training.
- Features are standardized using StandardScaler before fitting the SVM.
- All four kernels are compared under identical preprocessing.
- Hyperparameters (C, gamma, degree) are tuned with GridSearchCV using 5-fold stratified CV.
- RBF is the best kernel: test accuracy 0.9380, ROC-AUC 0.9756, F1 score 0.8364.
- 124 samples out of 2000 test rows are misclassified by the best (RBF) kernel.

CONCLUSION

An SVM classifier was implemented for breast cancer diagnosis using the Breast Cancer Prediction dataset. Data preprocessing – irrelevant feature removal, categorical one-hot encoding, training-based median imputation, and feature standardization – was applied before training. Four kernels (Linear, Polynomial, RBF, Sigmoid) were compared before and after hyperparameter tuning. Based on the test-set results, the RBF kernel with $C=1$ and $\text{gamma}=\text{scale}$ performed best with a testing accuracy of 0.938 and an ROC-AUC of 0.976, making it the most suitable choice for this classification task.

REFERENCES

- Scikit-learn Documentation – Support Vector Machines: <https://scikit-learn.org/stable/modules/svm.html>
- Scikit-learn Documentation – GridSearchCV: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
- Breast Cancer Prediction Dataset (Kaggle): [breast_cancer_prediction.csv](#)

LEARNING OUTCOMES

- I loaded, inspected, and visualized a real-world medical classification dataset using pandas and seaborn.
- I handled missing values and standardized features before training the SVM.
- I implemented SVM with four kernels (linear, polynomial, RBF, sigmoid).
- I used GridSearchCV with StratifiedKFold to tune C, gamma, and degree.
- I evaluated models using accuracy, precision, recall, F1 score, and ROC-AUC, and identified RBF as the best kernel.